Assignment 3: Data filtering using FIR filters

1 Reading in Data

1.1 Background

A doctor has measured an electrocardiogram, but for some reason, the signal is contaminated with high frequency noise. The doctor asks you whether you would be able to remove the high frequency noise from the signal so that he can more easily see the shape of the underlying signal.

You decide to try to remove the high frequency noise by using a simple FIR filter.

1.2 Problem specification

Write a Python *function* that does the following:

- Ask the user to select an in-file, which can have the extension `'.dat'`, `'.txt'`, or `'mat'`. Interactive selections can be done elegantly with the package `PySimpleGUI`. Since `PySimpleGUI` is missing in many Python distributions, you probably first have to install it from a *command terminal(!)* with

  ```
  pip install PySimpleGui
  ```

  The package can then be imported (in Python) using

  ```
  import PySimpleGUI as sg
  ```

  For the selection of a data file with specified file extensions, you can use

  ```
  file_name = sg.popup_get_file('Select the data-file',
                    no_window=True,
                    file_types=(('Data Files',
                                 ['*.dat', '*.mat', '*.txt']),))
  ```

- Determine the chosen extension.

  Tip: The class `Path` from the package `pathlib` provides many helpful functions to work with file-paths. For example with `file_name = Path('C:/Users/thomas')`, we can get the parent folder with `file.parent`, the stem with `file.stem`, and the extension with `file.suffix` (attention - the extension includes the ”.”!).

- If the user selects the extension `.dat` or `.txt`, load the data with `pd.read_csv`. If the file has the extension `.mat`, the file is a *Matlab* data file, and you can assume that the name of the loaded variable is `ecg_hfn`). Be careful to note that `pd.read_csv` returns a `DataFrame`! If the `DataFrame` is called `df`, the corresponding `np.array` can be accessed with `df.values`

- Return the data that have been read in.

2 Smoothing ECG-Data

2.1 Problem specification

- Load the data with the function described in *Ex 1*.

- Use a moving average filter to reduce the high frequency noise without changing the shape of the underlying signal.
- Use a Savitzky-Golay filter to reduce the high frequency noise without changing the shape of the underlying signal.
- Plot the raw data and the smoothed data on the same graph, so that it is possible to judge by eye how well the smoothing works.

3 Heartbeat and Differentiation

3.1 Background

The doctor is happy with the results of the filtering that you have performed. Now, however, the doctor would like to know what the average heart rate of the patient was.

One way of determining heart rate is to measure the time that passes between two R peaks.

3.2 Problem specification

- Ask the user to *interactively* select two adjacent R-peaks. This can be done with the command `plt.ginput(2)`. Use that information to estimate the sampling rate, and display this information.
- Hint: you can use `plt.title(...)` to tell the user what to do.
- Once you know the sampling rate, differentiate the data with a Savitzky-Golay filter and show the results.

4 Plotting Derivatives

- Use a Savitzky-Golay filter to estimate the first derivative of the ECG signal.
- Use a Savitzky-Golay filter to estimate the second derivative of the ECG signal.
- Show the smoothed data (i.e., the zeroth derivative) and the first and second derivatives, all in the same figure.
- Use the object-oriented plotting style

  ```
  fig, axs = plt.subplots(3, 1, sharex=True)
  ```

  and

  ```
  axs[..].plot(...)
  ```

  to simultaneously zoom in on all x-axes. If you are working in a *Jupyter* environment, first switch to an external graphics window with `%matplotlib`. You can switch back to inline display of graphics with `%matplotlib inline`.